

```
/* =====  
DBMS LAB PRACTICAL  
UNIVERSITY ACADEMIC MANAGEMENT SYSTEM  
===== */
```

```
/* STEP 1: CREATE DATABASE */  
DROP DATABASE IF EXISTS University_Academic_DB;  
CREATE DATABASE University_Academic_DB;  
USE University_Academic_DB;
```

```
/* =====  
STEP 2: DEPARTMENT MASTER  
Department -> Programme  
Department -> Faculty  
===== */
```

```
CREATE TABLE Department_Master (  
    Dept_ID INT PRIMARY KEY,  
    Dept_Name VARCHAR(100) NOT NULL UNIQUE,  
    HOD_Name VARCHAR(100) NOT NULL  
);
```

```
INSERT INTO Department_Master  
(Dept_ID, Dept_Name, HOD_Name)  
VALUES  
(1, 'Computer Science', 'Dr. Rajesh Kumar'),  
(2, 'Mechanical Engineering', 'Dr. Anil Sharma'),  
(3, 'Electrical Engineering', 'Dr. Priya Singh'),  
(4, 'Civil Engineering', 'Dr. Amit Verma'),  
(5, 'Electronics Engineering', 'Dr. Neha Gupta');
```

```
/* =====  
STEP 3: PROGRAMME MASTER  
Department -> Programme  
===== */
```

```
CREATE TABLE Programme_Master (  
    Programme_ID INT PRIMARY KEY,  
    Programme_Name VARCHAR(100) NOT NULL,  
    Duration INT NOT NULL CHECK (Duration > 0),  
    Dept_ID INT NOT NULL,
```

```
FOREIGN KEY (Dept_ID)
REFERENCES Department_Master(Dept_ID)
);
```

```
INSERT INTO Programme_Master
(Programme_ID, Programme_Name, Duration, Dept_ID)
VALUES
(101, 'B.Tech Computer Science', 4, 1),
(102, 'B.Tech Mechanical Engineering', 4, 2),
(103, 'B.Tech Electrical Engineering', 4, 3),
(104, 'B.Tech Civil Engineering', 4, 4),
(105, 'B.Tech Electronics Engineering', 4, 5);
```

```
/* =====
STEP 4: SEMESTER MASTER
Programme -> Semester
===== */
```

```
CREATE TABLE Semester_Master (
Semester_ID INT PRIMARY KEY,
Semester_No INT NOT NULL CHECK (Semester_No BETWEEN 1 AND 8),
Programme_ID INT NOT NULL,

FOREIGN KEY (Programme_ID)
REFERENCES Programme_Master(Programme_ID)
);
```

```
INSERT INTO Semester_Master
(Semester_ID, Semester_No, Programme_ID)
VALUES
(201, 1, 101),
(202, 2, 101),
(203, 3, 101),
(204, 4, 101),
(205, 5, 101);
```

```
/* =====
STEP 5: COURSE MASTER
Semester -> Course
===== */
```

```
CREATE TABLE Course_Master (
```

```
Course_ID INT PRIMARY KEY,  
Course_Name VARCHAR(100) NOT NULL,  
Credits INT NOT NULL CHECK (Credits BETWEEN 1 AND 6),  
Semester_ID INT NOT NULL,
```

```
FOREIGN KEY (Semester_ID)  
REFERENCES Semester_Master(Semester_ID)
```

```
);
```

```
INSERT INTO Course_Master  
(Course_ID, Course_Name, Credits, Semester_ID)  
VALUES  
(301, 'Database Management System', 4, 201),  
(302, 'Data Structures', 4, 201),  
(303, 'Operating Systems', 4, 202),  
(304, 'Computer Networks', 3, 202),  
(305, 'Software Engineering', 3, 203);
```

```
/* =====  
STEP 6: FACULTY MASTER  
Department -> Faculty  
===== */
```

```
CREATE TABLE Faculty_Master (  
Faculty_ID INT PRIMARY KEY,  
Faculty_Name VARCHAR(100) NOT NULL,  
Designation VARCHAR(50) NOT NULL,  
Dept_ID INT NOT NULL,  
  
FOREIGN KEY (Dept_ID)  
REFERENCES Department_Master(Dept_ID)  
);
```

```
INSERT INTO Faculty_Master  
(Faculty_ID, Faculty_Name, Designation, Dept_ID)  
VALUES  
(401, 'Dr. Suresh Kumar', 'Professor', 1),  
(402, 'Dr. Rakesh Singh', 'Associate Professor', 1),  
(403, 'Dr. Pooja Sharma', 'Assistant Professor', 1),  
(404, 'Dr. Manoj Gupta', 'Professor', 2),  
(405, 'Dr. Kavita Verma', 'Assistant Professor', 3);
```

```
/* =====  
STEP 7: STUDENT MASTER  
Programme -> Student  
===== */
```

```
CREATE TABLE Student_Master (  
    Student_ID INT PRIMARY KEY,  
    Student_Name VARCHAR(100) NOT NULL,  
    DOB DATE NOT NULL,  
    Email VARCHAR(100) NOT NULL UNIQUE,  
    Programme_ID INT NOT NULL,  
  
    FOREIGN KEY (Programme_ID)  
        REFERENCES Programme_Master(Programme_ID)  
);
```

```
INSERT INTO Student_Master  
(Student_ID, Student_Name, DOB, Email, Programme_ID)  
VALUES  
(601, 'Amit Kumar', '2005-04-12', 'amit@gmail.com', 101),  
(602, 'Rahul Singh', '2005-07-21', 'rahul@gmail.com', 101),  
(603, 'Priya Sharma', '2004-11-10', 'priya@gmail.com', 101),  
(604, 'Neha Gupta', '2005-02-18', 'neha@gmail.com', 102),  
(605, 'Rohan Verma', '2004-09-25', 'rohan@gmail.com', 103);
```

```
/* =====  
STEP 8: FACULTY-COURSE ALLOCATION  
Faculty <-> Course  
Many-to-Many Relationship  
===== */
```

```
CREATE TABLE Faculty_Course_Allocation (  
    Allocation_ID INT PRIMARY KEY,  
    Faculty_ID INT NOT NULL,  
    Course_ID INT NOT NULL,  
    Academic_Year VARCHAR(20) NOT NULL,  
  
    FOREIGN KEY (Faculty_ID)  
        REFERENCES Faculty_Master(Faculty_ID),  
  
    FOREIGN KEY (Course_ID)  
        REFERENCES Course_Master(Course_ID)  
);
```

```
INSERT INTO Faculty_Course_Allocation
(Allocation_ID, Faculty_ID, Course_ID, Academic_Year)
VALUES
(501, 401, 301, '2026-27'),
(502, 401, 302, '2026-27'),
(503, 402, 303, '2026-27'),
(504, 402, 304, '2026-27'),
(505, 403, 305, '2026-27'),
(506, 403, 301, '2026-27'),
(507, 401, 303, '2026-27'),
(508, 402, 305, '2026-27'),
(509, 403, 302, '2026-27'),
(510, 401, 304, '2026-27');
```

```
/* =====
STEP 9: STUDENT-COURSE REGISTRATION
Student <-> Course
Many-to-Many Relationship
===== */
```

```
CREATE TABLE Student_Course_Registration (
    Registration_ID INT PRIMARY KEY,
    Student_ID INT NOT NULL,
    Course_ID INT NOT NULL,
    Registration_Date DATE DEFAULT (CURRENT_DATE),

    FOREIGN KEY (Student_ID)
        REFERENCES Student_Master(Student_ID),

    FOREIGN KEY (Course_ID)
        REFERENCES Course_Master(Course_ID)
);
```

```
INSERT INTO Student_Course_Registration
(Registration_ID, Student_ID, Course_ID, Registration_Date)
VALUES
(701, 601, 301, '2026-08-01'),
(702, 601, 302, '2026-08-01'),
(703, 602, 301, '2026-08-02'),
(704, 602, 303, '2026-08-02'),
(705, 603, 302, '2026-08-03'),
(706, 603, 304, '2026-08-03'),
```

```
(707, 604, 303, '2026-08-04'),  
(708, 604, 305, '2026-08-04'),  
(709, 605, 301, '2026-08-05'),  
(710, 605, 304, '2026-08-05');
```

```
/* =====  
STEP 10: DISPLAY ALL TABLES  
===== */
```

```
SHOW TABLES;
```

```
/* =====  
STEP 11: DISPLAY ALL RECORDS  
===== */
```

```
SELECT * FROM Department_Master;
```

```
SELECT * FROM Programme_Master;
```

```
SELECT * FROM Semester_Master;
```

```
SELECT * FROM Course_Master;
```

```
SELECT * FROM Faculty_Master;
```

```
SELECT * FROM Student_Master;
```

```
SELECT * FROM Faculty_Course_Allocation;
```

```
SELECT * FROM Student_Course_Registration;
```

```
/* =====  
STEP 12: QUERY 1  
Display all programmes under a particular department  
===== */
```

```
SELECT  
    p.Programme_ID,  
    p.Programme_Name,  
    p.Duration,  
    d.Dept_Name
```

```
FROM Programme_Master p
JOIN Department_Master d
  ON p.Dept_ID = d.Dept_ID
WHERE d.Dept_ID = 1;
```

```
/* =====
STEP 13: QUERY 2
Display all semesters and courses under a programme
===== */
```

```
SELECT
  p.Programme_Name,
  s.Semester_No,
  c.Course_ID,
  c.Course_Name,
  c.Credits
FROM Programme_Master p
JOIN Semester_Master s
  ON p.Programme_ID = s.Programme_ID
JOIN Course_Master c
  ON s.Semester_ID = c.Semester_ID
WHERE p.Programme_ID = 101;
```

```
/* =====
STEP 14: QUERY 3
Display all students enrolled in a programme
===== */
```

```
SELECT
  s.Student_ID,
  s.Student_Name,
  s.DOB,
  s.Email,
  p.Programme_Name
FROM Student_Master s
JOIN Programme_Master p
  ON s.Programme_ID = p.Programme_ID
WHERE p.Programme_ID = 101;
```

```
/* =====
STEP 15: QUERY 4
```

Display all courses taught by a particular faculty

===== */

```
SELECT
  f.Faculty_ID,
  f.Faculty_Name,
  c.Course_ID,
  c.Course_Name,
  a.Academic_Year
FROM Faculty_Master f
JOIN Faculty_Course_Allocation a
  ON f.Faculty_ID = a.Faculty_ID
JOIN Course_Master c
  ON a.Course_ID = c.Course_ID
WHERE f.Faculty_ID = 401;
```

/* =====

STEP 16: QUERY 5

Display all students registered for a particular course

===== */

```
SELECT
  c.Course_ID,
  c.Course_Name,
  s.Student_ID,
  s.Student_Name,
  r.Registration_Date
FROM Course_Master c
JOIN Student_Course_Registration r
  ON c.Course_ID = r.Course_ID
JOIN Student_Master s
  ON r.Student_ID = s.Student_ID
WHERE c.Course_ID = 301;
```

/* =====

STEP 17: QUERY 6

Complete hierarchy

Department -> Programme -> Semester -> Course

===== */

```
SELECT
  d.Dept_Name,
```



```
p.Programme_Name,
s.Semester_No,
c.Course_ID,
c.Course_Name,
c.Credits
FROM Department_Master d
JOIN Programme_Master p
  ON d.Dept_ID = p.Dept_ID
JOIN Semester_Master s
  ON p.Programme_ID = s.Programme_ID
JOIN Course_Master c
  ON s.Semester_ID = c.Semester_ID
ORDER BY
  d.Dept_ID,
  p.Programme_ID,
  s.Semester_No;
```

```
/* =====
STEP 18: DEPARTMENT -> FACULTY
===== */
```

```
SELECT
  d.Dept_Name,
  f.Faculty_ID,
  f.Faculty_Name,
  f.Designation
FROM Department_Master d
JOIN Faculty_Master f
  ON d.Dept_ID = f.Dept_ID;
```

```
/* =====
STEP 19: PROGRAMME -> STUDENT
===== */
```

```
SELECT
  p.Programme_Name,
  s.Student_ID,
  s.Student_Name,
  s.Email
FROM Programme_Master p
JOIN Student_Master s
  ON p.Programme_ID = s.Programme_ID;
```

```
/* =====  
STEP 20: FACULTY -> COURSE  
===== */
```

```
SELECT  
    f.Faculty_Name,  
    c.Course_Name,  
    a.Academic_Year  
FROM Faculty_Master f  
JOIN Faculty_Course_Allocation a  
    ON f.Faculty_ID = a.Faculty_ID  
JOIN Course_Master c  
    ON a.Course_ID = c.Course_ID;
```

```
/* =====  
STEP 21: STUDENT -> COURSE  
===== */
```

```
SELECT  
    s.Student_Name,  
    c.Course_Name,  
    r.Registration_Date  
FROM Student_Master s  
JOIN Student_Course_Registration r  
    ON s.Student_ID = r.Student_ID  
JOIN Course_Master c  
    ON r.Course_ID = c.Course_ID;
```

```
/* =====  
STEP 22: CHECK TABLE STRUCTURES  
===== */
```

```
DESC Department_Master;
```

```
DESC Programme_Master;
```

```
DESC Semester_Master;
```

```
DESC Course_Master;
```

DESC Faculty_Master;

DESC Student_Master;

DESC Faculty_Course_Allocation;

DESC Student_Course_Registration;

```
/* =====  
STEP 23: COUNT RECORDS  
===== */
```

```
SELECT 'Department_Master' AS Table_Name,  
       COUNT(*) AS Total_Records  
FROM Department_Master
```

UNION ALL

```
SELECT 'Programme_Master',  
       COUNT(*)  
FROM Programme_Master
```

UNION ALL

```
SELECT 'Semester_Master',  
       COUNT(*)  
FROM Semester_Master
```

UNION ALL

```
SELECT 'Course_Master',  
       COUNT(*)  
FROM Course_Master
```

UNION ALL

```
SELECT 'Faculty_Master',  
       COUNT(*)  
FROM Faculty_Master
```

UNION ALL

```
SELECT 'Student_Master',
```

```
    COUNT(*)  
FROM Student_Master
```

```
UNION ALL
```

```
SELECT 'Faculty_Course_Allocation',  
       COUNT(*)  
FROM Faculty_Course_Allocation
```

```
UNION ALL
```

```
SELECT 'Student_Course_Registration',  
       COUNT(*)  
FROM Student_Course_Registration;
```

```
/* =====  
STEP 24: REFERENTIAL INTEGRITY TEST  
This query SHOULD produce a foreign-key error  
because Dept_ID = 999 does not exist.  
===== */
```

```
INSERT INTO Programme_Master  
(Programme_ID, Programme_Name, Duration, Dept_ID)  
VALUES  
(999, 'Test Programme', 4, 999);
```

```
/* =====  
END OF PRACTICAL  
===== */
```

Result grid:

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
Table_Name	Total_Records				
▶ Department_Master	5				
Programme_Master	5				
Semester_Master	5				
Course_Master	5				
Faculty_Master	5				
Student_Master	5				
Faculty_Course_Allocation	10				
Student_Course_Registration	10				